Drought Triggers Training Document

Release v0.1

-

Apr 08, 2025

Table of Contents

Anticipatory Action for Drought Early Warning

1.1 Concept of Anticipatory Action (AA)

Anticipatory Action (AA) represents a paradigm shift in humanitarian response, where actions are taken before a disaster occurs based on forecasts rather than after impacts materialize. In drought contexts, AA leverages the slow-onset nature of droughts to implement interventions during the critical "window of opportunity" between a reliable forecast and the onset of drought impacts.

According to Guimarães Nobre et al. (2023), AA enables the implementation of predefined actions triggered by forecasts that exceed specific thresholds, facilitating early risk reduction. This approach links seasonal forecasting information with predetermined contingency plans, actors, and funding—often referred to as Forecast-based Financing (FbF). Unlike traditional humanitarian responses that mobilize after a crisis, AA systems preposition resources and initiate interventions when forecast indicators reach trigger values, potentially reducing losses to livelihoods and infrastructure while providing more dignified support to communities.

1.2 Ensemble Prediction Systems

Ensemble Prediction Systems (EPS) form the backbone of modern seasonal forecasting, addressing inherent uncertainties in weather and climate prediction. These systems generate multiple forecasts (ensemble members) by varying initial conditions, model physics, or both. The European Centre for Medium-Range Weather Forecasts' SEAS5 system, for instance, provides 25-51 ensemble members with lead times up to 7 months.

As described by Guimarães Nobre et al. (2024), ensemble forecasts allow for probabilistic interpretation of potential outcomes, crucial for risk-based decision-making. Each ensemble member represents an equally likely future scenario, collectively providing information about forecast uncertainty and the range of possible outcomes. This probabilistic approach is essential for AA systems where decisions involve significant resource allocation under uncertainty.

1.3 Empirical Probability Calculation

For drought monitoring and forecasting, empirical probability calculations convert ensemble forecasts into decision-relevant metrics. The process involves:

1. Calculating drought indicators (e.g., Standardized Precipitation Index) for each ensemble member
2. Determining what proportion of ensemble members exceed predefined thresholds
3. Deriving empirical probabilities that represent the likelihood of drought conditions

These empirical probabilities become the basis for triggers in AA systems. According to Guimarães Nobre et al. (2023), the empirical probability approach enables quantitative assessment of forecast uncertainty, allowing decision-makers to balance the risks of action versus inaction.

1.4 Role of Long-term Verification

Long-term verification forms the cornerstone of trust and justification for any AA system. By systematically comparing historical forecasts with observed outcomes over many years, verification:

1. Quantifies forecast quality through metrics like hit rates and false alarm ratios
2. Establishes appropriate trigger values based on performance statistics
3. Builds confidence among stakeholders and donors in the AA approach
4. Supports continuous improvement of the forecasting system

As emphasized in both papers, verification analysis provides the evidence base needed to justify anticipatory investments. It ensures that actions are triggered at appropriate probability thresholds that balance the risks of missed events against false alarms—a critical consideration when resources are limited and each intervention must be carefully justified.

1.5 AUROC: A Key Metric for Forecast Verification

The Area Under the Receiver Operating Characteristic curve (AUROC) represents a fundamental metric in the verification of probabilistic forecasts for Anticipatory Action (AA) systems. This metric addresses a central question: does the forecasting system effectively discriminate between drought and non-drought events?

1.5.1 AUROC Concept and Calculation

The AUROC score quantifies a forecast's ability to discriminate between binary outcomes (drought /no drought) across different probability thresholds. As described by Guimarães Nobre et al. (2023), the calculation involves setting a range of probability thresholds for converting forecasts into binary predictions, then evaluating how well these predictions match observations.

The resulting curve plots the hit rate (proportion of observed droughts correctly forecast) against the false alarm rate (proportion of non-droughts incorrectly forecast as droughts) at various threshold levels. The area under this curve—the AUROC score—ranges from 0 to 1, where:

- 0.5 indicates no skill (equivalent to random guessing)
- 1.0 indicates perfect discrimination
- Values below 0.5 suggest systematically incorrect forecasts

1.6 Quality Threshold for Anticipatory Action

In operational AA systems for drought, an AUROC score of 0.5 serves as a critical quality threshold. As implemented in the "Ready, Set & Go!" system (Guimarães Nobre et al., 2024), forecasts with AUROC scores below 0.5 are considered unsuitable for triggering anticipatory actions, as they provide no added value over random chance.

This threshold represents a minimum standard for forecast quality—one that ensures actions are based on information with at least some predictive skill. When evaluating potential triggers for AA, only those derived from forecasts with AUROC > 0.5 are considered, creating a first-level quality control in the system design.

1.7 Bootstrapping for Uncertainty Assessment

To account for sampling uncertainty in AUROC estimation, the verification process employs bootstrapping techniques. By repeatedly resampling the forecast-observation pairs with replacement (typically 1,000 iterations), a distribution of AUROC scores is generated, allowing for:

1. Estimation of confidence intervals around the AUROC score
2. Assessment of statistical significance of skill above the 0.5 threshold
3. More robust decision-making about forecast quality

This approach acknowledges that verification statistics based on limited historical records contain inherent uncertainty, particularly for rare events like severe droughts.

1.8 Role in Trigger Selection

Beyond serving as a quality filter, AUROC scores influence the selection of optimal trigger values. Higher AUROC scores indicate greater potential for skillful discrimination, allowing for more confident selection of triggers based on other performance metrics like hit rates and false alarm ratios.

In the context of long-term verification, AUROC provides a foundation for justifying AA initiatives to stakeholders and funders by demonstrating that the underlying forecasting system possesses genuine predictive skill, making it a suitable basis for humanitarian decision-making.

1.9 References

1. Nobre, Gabriela Guimarães, et al. “Forecasting, thresholds, and triggers: towards developing a forecast-based financing system for droughts in Mozambique.” Climate Services 30 (2023): 100344.
2. Guimarães Nobre, Gabriela, et al. “Ready, Set & Go! An anticipatory action system against droughts.” Natural Hazards and Earth System Sciences 24.12 (2024): 4661-4682.

Chapter 2

Essential Tools

This document provides a brief introduction to the fundamental tools needed for developing Standardized Precipitation Index (SPI) forecasting systems and drought monitoring triggers: Python programming language, Git version control with GitHub, and Jupyter Notebooks for interactive analysis.

2.1 Introduction to Python

Python has become the programming language of choice for scientific computing and data analysis due to its versatility, readability, and extensive ecosystem of libraries.

Key Concepts:

- **Scientific Libraries:** Python offers powerful packages like NumPy, SciPy, and pandas for processing and analyzing large datasets. Libraries such as xarray excel at handling multidimensional labeled data common in environmental sciences.
- **Statistical Processing:** A comprehensive suite of statistical libraries enables everything from basic descriptive statistics to complex machine learning models and time series analysis.
- **Geospatial Analysis:** Python's well-developed GIS libraries (GeoPandas, Shapely, Rasterio) support sophisticated spatial operations, coordinate transformations, and integration with remote sensing data.

Python 3.8 or newer is recommended for scientific applications, as recent versions offer performance improvements and newer features. The Anaconda distribution (https://www.anaconda.com/products/distribution) provides a convenient way to install Python along with most commonly used scientific libraries in a single package.

2.2 Git and GitHub for Scientific Reproducibility

Version control is fundamental to scientific computing, providing the infrastructure for reproducible research, rigorous verification, and transparent collaboration.

Key Concepts:

- **Code Versioning:** Git creates a detailed history of all changes, enabling scientists to track the evolution of analysis methods and maintain provenance of results.
- **Reproducibility:** By capturing both code and configuration files at each stage, researchers can precisely recreate analytical environments and verify that results are consistent across different systems and time periods.
- **Scientific Collaboration:** GitHub facilitates peer review of scientific code, enables geographically distributed teams to work on shared datasets, and supports the open science movement through

public repositories.

- **Verification Workflows:** Through branching and pull requests, teams can implement rigorous verification protocols where new analytical methods or triggers must be independently tested before being incorporated into operational systems.

In fields where computational results inform policy decisions or emergency response, version control provides the audit trail necessary for scientific credibility and accountability. Git's ability to document exactly how and when analytical thresholds were developed makes it particularly valuable for creating verified trigger systems in early warning applications.

The current code base along with the source files to create the documentaiton in pdf is stored in github link (https://github.com/icpac-igad/ibf-thresholds-triggers/tree/kmj) where the kmj is the current branch name.

2.3 Jupyter Notebooks as a Python Learning Tool

Jupyter Notebooks provide an excellent environment for learning Python and developing data analysis skills through interactive computing.

Key Concepts:

- **Interactive Learning:** Notebooks enable immediate execution of code cells, allowing learners to experiment with Python concepts and see results instantly.
- **Code and Documentation Together:** The ability to combine executable code with explanatory text makes notebooks ideal for self-guided learning and tutorials.
- **Visualization Integration:** Built-in support for charts and graphs allows beginners to easily visualize their data without switching between different tools.
- **Incremental Development:** The cell-based structure encourages breaking problems into manageable steps, helping new programmers develop good coding practices.

Jupyter Notebooks lower the barrier to entry for Python beginners by providing a more intuitive interface than traditional IDEs. The immediate feedback loop accelerates the learning process and helps users build confidence in their programming skills.

To get started with Jupyter, install it via pip or conda:

```
pip install notebook
```

Then launch it from your terminal:

```
jupyter notebook
```

2.4 Python Scripts and Jupyter Notebooks

Python scripts form the backbone of data analysis workflows, while Jupyter Notebooks provide an interactive environment to develop and test these scripts.

Python Scripts:

- **Structure:** Python scripts are plain text files with a `.py` extension containing Python code that can be executed from start to finish.
- **Execution:** Scripts can be run from the command line using the Python interpreter:

```
python script_name.py
```

- **Modularity:** Well-designed scripts can be imported and reused across different projects,

promoting code organization and efficiency.

- **Automation:** Scripts can be scheduled to run automatically or integrated into data processing pipelines.

Jupyter Notebooks:

- **Script Development:** Notebooks excel as a development environment where you can write and test code before converting it to production scripts.
- **Cell-Based Execution:** Unlike scripts that run from top to bottom, notebooks allow executing individual cells in any order, helping identify and fix issues.
- **Converting Between Formats:** Notebooks can be converted to Python scripts using the `nbconvert` tool:

```
jupyter nbconvert --to python notebook.ipynb
```

- **Importing Scripts:** Python scripts can be imported into notebooks using standard import statements, enabling you to test functions from your scripts interactively.

2.5 Integration for Operational Drought Forecasting

These three technologies work together to create robust drought forecasting systems:

1. **Python** handles the core calculations for SPI forecasting, statistical verification, and trigger development
2. **Git/GitHub** ensures forecast model integrity, enables collaboration, and tracks methodological changes
3. **Jupyter Notebooks** facilitates exploratory analysis, verification testing, and creation of drought forecast bulletins

By mastering these complementary tools, meteorologists and hydrologists can develop more reliable, transparent, and actionable drought early warning systems that connect seasonal forecasts to humanitarian decision-making.

Chapter 3

Overview of Python Libraries for Forecast Verification

3.1 Core Libraries for Forecast Analysis

3.1.1 xarray

Essential for handling multidimensional arrays with labeled coordinates. Provides powerful data structures for working with forecast and observational gridded data across multiple time periods and ensemble members.

3.1.2 xclim

Specialized for climate indices calculation, particularly the standardized_precipitation_index function, which is fundamental for translating precipitation data into drought indicators using gamma distribution fitting.

3.1.3 xskillscore

Provides verification metrics specific to Earth science data. Used for calculating ROC curves and skill scores when evaluating forecast performance against observations.

3.1.4 xhistogram

Enables efficient histogram generation along multiple dimensions for gridded data. Critical for categorizing forecast and observation pairs into contingency tables for binary verification.

3.1.5 xesmf

Handles regridding between different spatial resolutions. Essential for aligning ECMWF SEAS5 forecast data with CHIRPS observational data before verification can begin.

3.1.6 xbootstrap

Implements bootstrapping methods for uncertainty quantification. Used to generate confidence intervals for AUROC scores and other verification metrics.

3.2 Statistical and Geospatial Processing

3.2.1 numpy

Provides the mathematical foundation for array operations. Handles the numerical calculations underpinning all verification metrics and statistical analyses.

3.2.2 pandas

Manages tabular data structures for storing verification results. Facilitates filtering, aggregating, and organizing trigger statistics by threshold level and lead time.

3.2.3 dask

Enables parallel computing and lazy evaluation for large datasets. Critical for handling the memory-intensive forecast ensemble calculations across multiple lead times.

3.2.4 regionmask

Creates masked arrays based on geographic boundaries. Essential for focusing the analysis on specific regions like Karamoja while excluding irrelevant grid cells.

3.2.5 geopandas

Extends pandas to work with geospatial data. Handles the shapefile operations needed to define region boundaries and create regional masks.

3.2.6 shapely

Provides computational geometry operations. Used for manipulating region polygons, calculating bounds, and applying buffers around analysis regions.

3.3 Visualization

3.3.1 matplotlib

Creates the foundational plots and visualizations. Generates time series, heatmaps, and spatial maps of SPI values and forecast skill.

3.3.2 seaborn

Builds on matplotlib to create more attractive statistical visualizations. Particularly useful for the heatmaps showing hit rates and false alarm ratios.

3.3.3 altair

Provides a declarative visualization syntax. Used for creating interactive and linked visualization components showing forecast performance.

3.3.4 vl_convert

Converts Vega-Lite specifications (from Altair) to static image formats like PNG. Essential for including visualizations in reports.

3.4 Cloud Data Access

3.4.1 google.oauth2

Manages authentication for accessing cloud-stored data. Enables secure access to forecast and observation datasets stored in Google Cloud Storage.

3.4.2 fsspec

Provides a unified file system interface. Allows consistent access to data regardless of whether it's stored locally or in cloud storage.

These libraries work together to form a comprehensive toolkit for drought forecast verification, enabling the entire workflow from data retrieval through processing, verification, and visualization of results.

Setting Up MicroMamba and Accessing Jupyter Notebook Remotely

This guide will walk you through setting up MicroMamba and accessing Jupyter Notebook from a remote computer using MobaXterm, a popular terminal client for Windows users.

4.1 Part 1: Setting Up MicroMamba

4.1.1 Installing MicroMamba

Linux/macOS

1. Open your terminal
2. Run the following command to install MicroMamba:

```
"${SHELL}" <(curl -L micro.mamba.pm/install.sh)
```

3. Follow the on-screen prompts to complete the installation
4. Close and reopen your terminal, or run `source ~/.bashrc` to apply the changes

Windows

MicroMamba also has Windows support. For Windows, we recommend using PowerShell:

1. Open PowerShell
2. Run the following commands:

```
Invoke-Webrequest -URI https://micro.mamba.pm/api/micromamba/win-64/latest
 -OutFile micromamba.tar.bz2
tar xf micromamba.tar.bz2

MOVE -Force Library\bin\micromamba.exe micromamba.exe
.\micromamba.exe --help

# You can use e.g. $HOME\micromambaenv as your base prefix
$Env:MAMBA_ROOT_PREFIX="C:\Your\Root\Prefix"

# Invoke the hook
.\micromamba.exe shell hook -s powershell | Out-String | Invoke-Expression

# ... or initialize the shell
.\micromamba.exe shell init -s powershell -r C:\Your\Root\Prefix
```

4.1.2 Creating an Environment

1. If you have an environment file (environment.yml):

```
micromamba create -f environment.yml
```

2. If you don't have an environment file, you can create one with specific packages:

```
micromamba create -n drought_env python==3.12
```

4.1.3 Activating the Environment

```
micromamba activate drought_env
```

4.1.4 Installing Additional Packages

1. Install packages from conda-forge channel:

```
micromamba install netcdf4 -c conda-forge
```

2. Install packages using pip:

```
pip install xclim==0.56.0
```

4.2 Part 2: Remote SSH Access - For Windows and Linux Users

4.2.1 For Windows Users: Using MobaXterm

MobaXterm is a powerful terminal emulator for Windows that includes SSH, SFTP, and X11 forwarding capabilities.

1. **Install MobaXterm**:
   - Download from MobaXterm website
   - Choose either Installer edition (recommended) or Portable edition
   - Follow the installation wizard if using the Installer edition
2. **Create an SSH Connection**:
   - Launch MobaXterm
   - Click on "Session" in the top-left corner
   - Select "SSH" from the available session types
   - Enter your remote server details: - Remote host: `your_remote_server_ip` - Username: `your_username` - Port: 22 (default)
   - Optionally configure SSH key authentication in the "Advanced SSH settings" tab
   - Save your session for future use
3. **Set Up Port Forwarding for Jupyter**:
   - While creating your SSH session, click on the "Tunneling" tab
   - Click "Add a new port forwarding"
   - Configure: - Forward port: 4888 (local machine) - Remote server: localhost - Remote port: 4888 (remote machine)

- Save these settings with your session

4. **Connect to Your Server**:
   - Click on your saved session in the left sidebar
   - Enter your password when prompted (if not using key authentication)
   - You now have terminal access to your remote server
5. **Additional MobaXterm Features**:
   - Built-in SFTP browser in the left panel for easy file transfers
   - Multiple tabs for working with several connections
   - Saved sessions for quick access to your servers

4.2.2 For Linux Users: Using Terminal

Linux systems come with built-in terminal and SSH capabilities.

1. **Open Terminal**:
   - Use your system's application launcher or press Ctrl+Alt+T
2. **Connect to Remote Server**:
   - Use the SSH command:

```
ssh username@remote_machine_ip
```

   - Enter your password when prompted
3. **Set Up SSH Keys for Passwordless Login** (optional but recommended):
   - Generate an SSH key pair (if you don't already have one):

```
ssh-keygen -t rsa -b 4096
```

   - Copy your public key to the remote server:

```
ssh-copy-id username@remote_machine_ip
```

   - Now you can connect without entering a password
4. **Set Up Port Forwarding for Jupyter**:
   - When connecting, add the port forwarding parameter:

```
ssh -L 4888:localhost:4888 username@remote_machine_ip
```

   - This creates a secure tunnel from your local port 4888 to the remote server's port 4888
5. **Using Terminal Multiplexers** (recommended):
   - Install and use tmux or screen for session persistence:

```
# Install tmux
sudo apt-get install tmux  # For Debian/Ubuntu
# or
sudo yum install tmux      # For RedHat/CentOS

# Start a tmux session
tmux

# Detach from session (keeps running in background)
```

```
# Press Ctrl+b then d

# Reattach to session
tmux attach
```

Both Windows and Linux users will follow the same remaining steps for activating the MicroMamba environment and starting Jupyter Notebook as described in Part 3. Please refer to the attached "Using MobaXterm for Remote Access and Jupyter Notebook" guide for detailed instructions with visual references.

4.3 Part 3: Setting Up Jupyter Notebook for Remote Access

4.3.1 Starting Jupyter Lab on the Remote Machine

1. Connect to your remote machine using MobaXterm as described in the attached guide
2. Activate your environment:

   ```
   micromamba activate drought_env
   ```

3. Start Jupyter Lab specifying a port (e.g., 4888):

   ```
   jupyter lab --no-browser --port=4888
   ```

4. You'll see output containing a URL with a token. It will look something like:

   ```
   http://localhost:4888/?token=abcdef123456...
   ```

   Note this URL for the next step.

4.3.2 Accessing Jupyter Lab from Your Local Browser

1. Open a web browser on your Windows computer
2. Paste the URL with the token from step 1:

   ```
   http://localhost:4888/?token=abcdef123456...
   ```

   or simply go to:

   ```
   http://localhost:4888
   ```

   and enter the token when prompted.

4.4 Troubleshooting Tips

4.4.1 Port Already in Use

If port 4888 is already in use, choose a different port:

```
jupyter lab --no-browser --port=4889
```

Remember to adjust your port forwarding in MobaXterm accordingly.

4.4.2 Connection Issues

- Make sure your firewall allows connections to the specified port

- Verify that the Jupyter server is actually running
- Check if another Jupyter instance is already using that port

4.4.3 Session Disconnects

Configure SSH keepalive in MobaXterm:

- Go to Settings → SSH
- Set "SSH keepalive" to a value like 30 seconds

4.5 Setting Up a Password for Jupyter

For easier access in the future:

1. Set up a password:

```
jupyter server password
```

2. Enter and confirm your password
3. Start Jupyter as usual, and now you can use your password instead of the token

4.6 Using Your Remote Jupyter Environment

1. Your files and data on the remote machine will be accessible through the Jupyter interface
2. Any packages installed in your `drought_env` environment will be available in notebooks
3. Use MobaXterm's SFTP browser (left sidebar) to easily upload/download files between your local and remote machines

For more detailed instructions on using MobaXterm for SSH connections and managing remote Jupyter sessions, please refer to the attached documentation.

Acquiring Data for Drought Forecasting: CHIRPS and ECMWF SEAS5

This guide explains how to download the two essential datasets required for drought forecasting using Standardized Precipitation Index (SPI):

1. **CHIRPS** - Climate Hazards Group InfraRed Precipitation with Station data (observational dataset)
2. **ECMWF SEAS5** - Seasonal forecast system from the European Centre for Medium-Range Weather Forecasts

5.1 Downloading CHIRPS Observational Data

CHIRPS version 3.0 provides quasi-global rainfall data from 1981 to near-present, making it ideal for drought monitoring.

Data Access Options:

1. **Direct Web Download**
   CHIRPS data can be downloaded directly from the Climate Hazards Center website:

   ```
   https://data.chc.ucsb.edu/products/CHIRPS/v3.0/
   ```

2. **Python Implementation with wget**

```python
import os
import subprocess

def download_chirps(output_dir="./data", filename="chirps-v3.0.monthly.nc"):
    """
    Download CHIRPS precipitation data using wget

    Args:
        output_dir: Directory to save the downloaded data
        filename: Name for the output file

    Returns:
        str: Path to the downloaded file
    """
    print("Downloading CHIRPS precipitation data...")

    # Create output directory if it doesn't exist
    os.makedirs(output_dir, exist_ok=True)

    output_file = os.path.join(output_dir, filename)
    url = "https://data.chc.ucsb.edu/products/CHIRPS/v3.0/monthly/global
/netcdf/chirps-v3.0.monthly.nc"
```

```
    try:
        # Check if file already exists
        if os.path.exists(output_file):
            print(f"CHIRPS data already exists at: {output_file}")
            return output_file

        # Use wget to download the file
        subprocess.run(["wget", url, "-O", output_file], check=True)
        print(f"CHIRPS data downloaded successfully to: {output_file}")
        return output_file
    except subprocess.CalledProcessError as e:
        print(f"Error downloading CHIRPS data: {e}")
        return None
    except Exception as e:
        print(f"Unexpected error: {e}")
        return None

# Example: Download CHIRPS data
chirps_file = download_chirps()

# Open the data with xarray
import xarray as xr
ds = xr.open_dataset(chirps_file)
print(ds)
```

5.2 ECMWF SEAS5 Forecast Data from the Climate Data Store (CDS)

SEAS5 is ECMWF's fifth generation seasonal forecast system, providing global forecasts with lead times up to 7 months.

CDS Account Creation:

1. **Register for a CDS Account**
   Visit the Copernicus Climate Data Store (CDS) website:

   ```
   https://cds.climate.copernicus.eu/user/register
   ```

   Complete the registration form and agree to the Terms and Conditions.

2. **Generate an API Key**
   After registration:

   - Log in to your CDS account
   - Go to your profile page (click on your username in the top-right corner)
   - Scroll down to the "API key" section
   - The page will display your User ID and API Key

3. **Configure the CDS API Client**
   Create a file called `.cdsapirc` in your home directory with the following contents:

   ```
   $ cat ~/.cdsapirc
   url: https://cds.climate.copernicus.eu/api/v2
   key: <your-user-id>:<your-api-key>
   verify: 0
   ```

   Replace `<your-user-id>` and `<your-api-key>` with the values from your profile.

Python Implementation for ECMWF SEAS5 Download:

```
import os
import datetime
import cdsapi
import glob

def download_seas5(output_dir="./data", filename_prefix="seas5_precipitation_"):
    """
    Download SEAS5 dataset from ECMWF CDS API

    Args:
        output_dir: Directory to save the downloaded data
        filename_prefix: Prefix for the output filename

    Returns:
        str: Path to the downloaded file
    """
    print("Downloading SEAS5 data from ECMWF CDS...")

    # Create output directory if it doesn't exist
    os.makedirs(output_dir, exist_ok=True)

    # Current date to use in the filename
    current_date = datetime.datetime.now().strftime("%Y%m%d")
    output_file = os.path.join(output_dir,
 f"{filename_prefix}{current_date}.grib")

    # Define the SEAS5 dataset and request parameters
    dataset = "seasonal-monthly-single-levels"
    request = {
        "originating_centre": "ecmwf",
        "system": "51",
        "variable": ["total_precipitation"],
        "year": [
            "1981", "1982", "1983", "1984", "1985", "1986", "1987", "1988",
 "1989",
            "1990", "1991", "1992", "1993", "1994", "1995", "1996", "1997",
 "1998",
            "1999", "2000", "2001", "2002", "2003", "2004", "2005", "2006",
 "2007",
            "2008", "2009", "2010", "2011", "2012", "2013", "2014", "2015",
 "2016",
            "2017", "2018", "2019", "2020", "2021", "2022", "2023", "2024", "2025"
        ],
        "month": [
            "01", "02", "03", "04", "05", "06",
            "07", "08", "09", "10", "11", "12"
        ],
        "leadtime_month": ["1", "2", "3", "4", "5", "6"],
        "data_format": "grib",
        "product_type": ["monthly_mean"],
        "area": [23, 21, -12, 53]  # North, West, South, East
    }

    try:
        client = cdsapi.Client()
        client.retrieve(dataset, request, output_file)
        print(f"SEAS5 data downloaded successfully to: {output_file}")
        return output_file
    except Exception as e:
        print(f"Error downloading SEAS5 data: {e}")
        return None
```

```
def cleanup_old_seas5_files(output_dir="./data",
 filename_prefix="seas5_precipitation_", keep_latest=True):
    """
    Remove old SEAS5 files, optionally keeping the latest one

    Args:
        output_dir: Directory containing the SEAS5 files
        filename_prefix: Prefix of the SEAS5 files
        keep_latest: Whether to keep the latest file
    """
    print("Cleaning up old SEAS5 files...")

    # Get all SEAS5 files
    pattern = os.path.join(output_dir, f"{filename_prefix}*.grib")
    files = glob.glob(pattern)

    if not files:
        print("No SEAS5 files found for cleanup.")
        return

    if keep_latest and len(files) > 1:
        # Sort files by modification time (newest last)
        files.sort(key=os.path.getmtime)
        # Remove the latest file from the list
        latest_file = files.pop()
        print(f"Keeping latest file: {os.path.basename(latest_file)}")

    # Remove remaining files
    for file in files:
        try:
            os.remove(file)
            print(f"Removed: {os.path.basename(file)}")
        except Exception as e:
            print(f"Error removing {file}: {e}")

# Example: Download SEAS5 data and clean up old files
seas5_file = download_seas5()
cleanup_old_seas5_files()

# Open the data with xarray
import xarray as xr
ds = xr.open_dataset(seas5_file)
print(ds)
```

Important CDS Usage Notes:

- There are **usage limits** for data retrieval from CDS (number of requests, volume)
- Large requests are **queued** and may take time to complete
- If your request fails, the system will automatically retry
- Monitor your pending requests in the CDS web interface under "My Requests"

5.3 Data Management and Processing

The provided script includes several important features for managing the downloaded data:

1. **Automatic File Management**:
    - CHIRPS v3.0 data is downloaded only once (checks if file already exists)

- SEAS5 files are automatically date-stamped and old files are cleaned up
- Option to keep all SEAS5 files if needed for historical analysis

2. **Regional Specification**:
   - SEAS5 data is downloaded for a specific region ([23°N, 21°E, -12°S, 53°E])
   - This focused approach reduces download size and processing time

3. **Preprocessing Considerations**:
   After downloading both datasets, you'll typically need to:
   - **Convert GRIB to NetCDF**: SEAS5 data is downloaded in GRIB format and may need conversion
   - **Handle Different Spatial Resolutions**: CHIRPS and SEAS5 have different resolutions
   - **Apply Temporal Alignment**: Match forecast periods with observational data

```
import xarray as xr
import cfgrib
import xesmf as xe

# Load datasets (note different formats)
forecast = xr.open_dataset("./data/seas5_precipitation_20250407.grib",
 engine="cfgrib")
obs = xr.open_dataset("./data/chirps-v3.0.monthly.nc")

# Define the regridder if needed
regridder = xe.Regridder(
    forecast,
    obs,
    "bilinear",
    periodic=True
)

# Apply regridding
forecast_regridded = regridder(forecast)
```

This automated approach to data acquisition forms the foundation for developing robust drought monitoring applications based on the Standardized Precipitation Index (SPI).

Chapter 6

SPI Calculation using Python Libraries

6.1 The Standardized Precipitation Index (SPI)

The Standardized Precipitation Index (SPI) serves as a primary drought indicator in the anticipatory action system, quantifying rainfall anomalies across different timescales. Originally developed by McKee et al. (1993), SPI represents how current precipitation compares to historical conditions, expressed in standard deviation units.

In the implementation found in `run-process-spi.py`, SPI is calculated for 3-month accumulation periods (SPI-3), capturing seasonal-scale rainfall anomalies particularly relevant to agricultural drought:

```
spi_3 = standardized_precipitation_index(
    aa,
    freq="MS",
    window=3,
    dist="gamma",
    method="APP",
    cal_start='1991-01-01',
    cal_end='2018-01-01',
    fitkwargs={"floc": 0}
)
```

SPI values typically range from -4 to +4, with negative values indicating precipitation deficits. The interpretation follows standard thresholds, where values below -1 represent severe drought conditions that occur approximately once every 6-7 years, making them suitable targets for anticipatory action.

6.2 The xclim Library: Climate Indices Made Accessible

The xclim library provides a robust framework for climate data analysis, with particular strengths in calculating standardized climate indices like SPI. Developed to work seamlessly with xarray data structures, xclim implements climate indices following scientific standards while addressing computational challenges associated with large climate datasets. The details of the method is avialable at(https://xclim.readthedocs.io/en/v0.39.0 /xclim.indicators.atmos.html#xclim.indicators.atmos._precip.standardized_precipitation_index) For drought monitoring, xclim's `standardized_precipitation_index` function offers several critical features:

1. **Distribution Selection**: While gamma distribution is the standard choice for SPI calculation (used in our implementation), xclim supports alternative distributions including Pearson Type III and empirical distributions.

2. **Calculation Methods**: The library supports multiple calculation approaches: - `"APP"`: Approximation method (used in our implementation) - `"ML"`: Maximum likelihood estimation - `"PWM"`: Probability weighted moments
3. **Calibration Period Control**: As seen in the implementation, the function allows for specific calibration periods (`cal_start` and `cal_end`), crucial for establishing a consistent reference climatology across both forecast and observation datasets.
4. **Zero Precipitation Handling**: The `fitkwargs={"floc": 0}` parameter ensures that gamma distribution fitting properly handles zero precipitation values, which can be common in arid regions.

6.3 The xarray Foundation: Multi-dimensional Data Structures

Underlying both the data processing and SPI calculation is the xarray library, which provides labeled multi-dimensional arrays designed specifically for scientific datasets. In the context of drought monitoring, xarray offers several advantages:

1. **Self-describing Data**: Coordinates, dimensions, and metadata travel with the data, simplifying the tracking of spatial and temporal information.
2. **Advanced Indexing**: The implementation leverages xarray's powerful selection capabilities (e.g., `.sel(lead=fm)`, `.sel(latitude=slice(lat_max, lat_min))`) for intuitive data subsetting.
3. **Parallel Computing Integration**: Through dask integration, xarray enables scalable processing of large ensembles, evident in the `.compute()` calls throughout the processing pipeline.
4. **Interoperability**: The seamless handling of different data formats (NetCDF, GRIB, Zarr) facilitates the integration of observation data (CHIRPS) with forecast data (SEAS5).

In the SPI calculation for ensemble forecast data, the processing handles ensemble members differently based on their characteristics. For older members (0-24), SPI is calculated using the 1991-2018 calibration period, while newer operational members (25-51) use a 2017-2024 calibration period, addressing the challenge of maintaining consistent statistical properties across the full ensemble.

The script `01-run-process-spi.py` performs four main operations:

1. Calculates SPI on CHIRPS observation data
2. Calculates SPI on SEAS51 forecast data
3. Regrids both datasets to a common grid
4. Applies a geographic mask to focus on a specific region

6.4 1. SPI Calculation on CHIRPS Data

The `process_chirps_data()` function handles the processing of CHIRPS observation data:

```
def process_chirps_data(region_id, credentials, extent, chirps_file=None,
 output_dir='.', res=0.25):
    # Load CHIRPS data either from local file or GCS storage
    if chirps_file:
        logger.info(f"Using local CHIRPS file: {chirps_file}")
        ds = xr.open_dataset(chirps_file)
    else:
        logger.info(f"Loading CHIRPS data from GCP")
        uri = 'gs://seas51/ea_chirps_v3_monthly_20250312.zarr'
        ds = xr.open_dataset(uri, engine='zarr', consolidated=False,
```

```
storage_options={'token': credentials})

    # Subset to the region of interest
    ds_subset = ds.sel(latitude=slice(lat_min, lat_max), longitude=slice(lon_min,
lon_max))

    # Regrid to regular grid
    ds1 = ds_subset_computed.rename({'longitude': 'lon', 'latitude': 'lat'})
    dr = ds1["precip"]

    # Calculate SPI-3
    spi_3 = standardized_precipitation_index(
        aa,
        freq="MS",
        window=3,
        dist="gamma",
        method="APP",
        cal_start='1991-01-01',
        cal_end='2018-01-01',
        fitkwargs={"floc": 0}
    )
```

Key points:

- Uses xarray to handle climate data
- Applies regional subsetting to focus on the area of interest
- Calculates SPI-3 (3-month Standardized Precipitation Index) using gamma distribution
- Uses a calibration period from 1991-2018

6.5 2. SPI Calculation on SEAS51 Forecast Data

The `process_seas51_data()` function processes forecast data from ECMWF SEAS51:

```
def process_seas51_data(region_id, obs_file, credentials, extent,
 seas51_file=None, output_dir='.'):
    # Load SEAS51 data from file or GCS
    if seas51_file:
        logger.info(f"Using local SEAS51 file: {seas51_file}")
        sds = xr.open_dataset(seas51_file, engine='cfgrib',
 backend_kwargs=dict(time_dims=('forecastMonth', 'time')))
    else:
        logger.info(f"Loading SEAS51 data from GCP")
        suri = 'gs://seas51/ea_seas51_20250312_v3.zarr'
        sds = xr.open_dataset(suri, engine='zarr', consolidated=False,
 storage_options={'token': credentials})

    # Subset to region
    sds_subset = sds.sel(latitude=slice(lat_min, lat_max),
 longitude=slice(lon_min, lon_max))

    # Calculate SPI-3 for each lead time (1-6 months)
    all_leads = []
    for lead_val in range(1, 7):
        # Process each lead time
        cont_spi = vt_apply_spi3_with_parameter_transfer(sds_subset_computed,
 lead_val)
        lead_data = xr.concat(cont_spi, dim='member')
        all_leads.append(lead_data)
```

Key points:

- Processes multiple ensemble members (0-51) from the SEAS51 seasonal forecast
- Handles different lead times (1-6 months) separately
- Special handling for members 0-24 vs 25-51 with different calibration periods
- Concatenates multiple ensemble members into a single dataset

6.6 3. Regridding Operations

Both datasets need to be on the same grid for comparison. The script includes regridding operations:

```
# For CHIRPS data:
ds_out = xr.Dataset({
    "lat": (["lat"], np.arange(lat_min, lat_max, res), {"units":
 "degrees_north"}),
    "lon": (["lon"], np.arange(lon_min, lon_max, res), {"units": "degrees_east"}),
})
regridder = xe.Regridder(ds1, ds_out, "conservative")
dr_out = regridder(dr, keep_attrs=True)

# For SEAS51 data (to match CHIRPS grid):
ds_out = xr.Dataset({
    "lat": (["lat"], kn_obs['lat'].values, {"units": "degrees_north"}),
    "lon": (["lon"], kn_obs['lon'].values, {"units": "degrees_east"}),
})
regridder = xe.Regridder(gd2, ds_out, "bilinear", periodic=False)
dr_out = regridder(agd, keep_attrs=True)
```

Key points:

- Uses xESMF for regridding operations
- CHIRPS data uses conservative regridding to a regular grid
- SEAS51 forecast data is regridded to match the CHIRPS observations grid using bilinear interpolation
- Regular grid uses the specified resolution (default 0.25°)

6.7 4. Masking Operations

The `mask_netcdf_with_shapefile()` function applies a geographic mask to both datasets:

```
def mask_netcdf_with_shapefile(forecast_path, obs_path, shapefile_df,
 buffer_size=0.25,
                               output_forecast_path='masked_forecast.nc',
                               output_obs_path='masked_obs.nc'):
    # Load datasets
    forecast_ds = xr.open_dataset(forecast_path)
    obs_ds = xr.open_dataset(obs_path)

    # Create buffered geometry if needed
    buffered_gdf = gdf.copy()
    if buffer_size > 0:
        buffered_gdf['geometry'] = gdf.geometry.buffer(buffer_size)

    # Create mask from shapefile
    f_mask = regionmask.mask_geopandas(buffered_gdf.geometry, f_lons, f_lats)
    f_bool_mask = ~np.isnan(f_mask)
```

```
    # Expand mask to match dataset dimensions
    f_expanded_mask = f_bool_mask.expand_dims({
        "lead": forecast_ds.lead,
        "member": forecast_ds.member,
        "init": forecast_ds.init
    })

    # Apply mask to set values outside region to NaN
    masked_forecast = forecast_ds.copy(deep=True)
    for var in forecast_ds.data_vars:
        masked_forecast[var] = forecast_ds[var].where(f_expanded_mask, np.nan)
```

Key points:

- Uses regionmask to create masks from GeoDataFrame
- Applies a buffer around the region to include border areas
- Expands the 2D mask to match the dimensionality of each dataset
- Sets values outside the mask region to NaN
- Creates masked versions of both the observations and forecasts

6.8 Main Execution Flow

The script's main execution follows this sequence:

```
if __name__ == "__main__":
    # Define region and get extent
    region_id ='kmj'
    gdf, extent = get_region_bounds(region_id, use_local=True)

    # Process CHIRPS data
    obs_file = process_chirps_data(
        region_id, credentials, extent, chirps_file=chirps_file
    )

    # Process SEAS51 data
    fct_file = process_seas51_data(
        region_id, obs_file, credentials, extent, seas51_file=seas51_file
    )

    # Apply masking
    masked_fct, masked_obs = mask_netcdf_with_shapefile(
        forecast_path=fct_file,
        obs_path=obs_file,
        shapefile_df=shapefile_df,
        buffer_size=0.25
    )
```

This comprehensive processing pipeline takes raw climate data and prepares it for drought forecasting analysis by:

1. Loading and subsetting the data to the region of interest
2. Computing SPI values for both observations and forecasts
3. Ensuring both datasets are on the same spatial grid
4. Applying geographic masks to focus only on the relevant area

The resulting NetCDF files can then be used for further analysis and visualization of drought risks and forecasting skill assessment.

Chapter 7

Regridding Datasets with xESMF

7.1 Spatial Resolution Harmonization

A critical step in comparing forecasts with observations involves addressing the inherent resolution mismatch between datasets. In the drought anticipatory action system, CHIRPS observations (native resolution of 0.05°) and SEAS5 forecasts (native resolution of 1.0°) must be harmonized to a common grid to enable meaningful comparisons. The `run-process-spi.py` script implements this harmonization through regridding, bringing both datasets to an intermediate resolution of 0.25° (approximately 25 km at the equator).

The regridding process handles two distinct cases:

1. **CHIRPS Observation Regridding (5 km → 25 km)**:

```
# Create output grid with regular spacing
ds_out = xr.Dataset({
    "lat": (["lat"], np.arange(lat_min, lat_max, res), {"units":
 "degrees_north"}),
    "lon": (["lon"], np.arange(lon_min, lon_max, res), {"units":
 "degrees_east"}),
})

# Regrid using conservative method
regridder = xe.Regridder(ds1, ds_out, "conservative")
dr_out = regridder(dr, keep_attrs=True)
```

   This upscaling process preserves precipitation totals through conservative regridding, which maintains the area-integrated rainfall quantity during the transition to a coarser grid.

2. **SEAS5 Forecast Regridding (100 km → 25 km)**:

```
# Create output grid matching observations
ds_out = xr.Dataset({
    "lat": (["lat"], kn_obs['lat'].values, {"units": "degrees_north"}),
    "lon": (["lon"], kn_obs['lon'].values, {"units": "degrees_east"}),
})

# Perform regridding
regridder = xe.Regridder(gd2, ds_out, "bilinear", periodic=False)
dr_out = regridder(agd, keep_attrs=True)
```

   For the forecast data, bilinear interpolation is employed to downscale from 100 km to 25 km, producing smoother transitions between grid cells appropriate for the forecast uncertainty.

7.2 The xESMF Library

The Earth System Modeling Framework (ESMF) regridding functionality is made accessible through xESMF, a Python interface that integrates seamlessly with xarray datasets. This library provides multiple interpolation methods appropriate for different variables and scientific needs:

1. **Conservative**: Preserves the integral of the field (used for CHIRPS precipitation upscaling)
2. **Bilinear**: Weighted average of nearest grid points (used for SEAS5 interpolation)
3. **Nearest-neighbor**: Assigns value from nearest source point
4. **Higher-order patch**: Smoother interpolation useful for continuous fields

In the implementation, regridding objects are created once and stored for repeated application across dimensions:

```
regridder = xe.Regridder(gd2, ds_out, "bilinear", periodic=False)
```

The `periodic=False` parameter indicates that the longitude dimension is not treated as periodic (wrapped around the globe), which is appropriate for regional domains in East Africa.

7.3 Handling Special Cases in Regridding

The regridding implementation addresses several technical challenges:

1. **Coordinate System Compatibility**: The script ensures coordinate naming consistency before regridding:

```
# Rename coordinates for consistency if needed
if 'longitude' in ds_p_m1.dims and 'latitude' in ds_p_m1.dims:
    gd2 = ds_p_m1.rename({'longitude': 'lon', 'latitude': 'lat'})
```

2. **Land-Sea Mask Preservation**: The regridding process preserves missing values to maintain the integrity of land-sea boundaries.
3. **Ensemble Dimension Handling**: When processing SEAS5 ensemble data, regridding is applied individually to each lead time to manage memory efficiently:

```
for fm in range(min(6, len(all_leads))):
    # Regridding for each lead time
    # ...
```

This approach ensures that both datasets align on a common 0.25° grid, enabling direct comparison between forecasts and observations while balancing spatial detail against computational efficiency in the SPI calculation and verification process.

Aligning Forecast and Observation Data with climpred

8.1 Time Alignment Challenge in Forecast Verification

One of the most complex challenges in verifying seasonal forecasts is aligning the time coordinates between forecasts and observations. The `make_obs_fct_dataset` function in `vthree_utils.py` addresses this critical task by integrating the SEAS5 forecast and CHIRPS observation datasets with different temporal structures:

```
def make_obs_fct_dataset(params):
    """
    Prepares observed and forecasted dataset subsets for a specific region,
    season, and lead time by aligning their temporal coordinates.
    """
```

The function performs a sophisticated alignment process that accounts for three distinct time dimensions in seasonal forecasting:

1. **Initialization Time (init)**: When the forecast was issued
2. **Lead Time (lead)**: How many months into the future the forecast extends
3. **Valid Time**: When the forecast is valid for (init + lead)

Since CHIRPS observations have only a `time` dimension while SEAS5 forecasts have both `init` and `forecastMonth` (lead time), direct comparison requires creating a compatible coordinate system.

8.2 The climpred Library

The alignment utilizes `climpred`, a specialized Python library for seasonal forecast verification that bridges the gap between forecast and observational data structures:

```
hindcast = HindcastEnsemble(a_fc)
hindcast = hindcast.add_observations(a_obs)
a_fc1 = hindcast.get_initialized()
```

This sequence of operations performs several crucial tasks:

1. Creates a `HindcastEnsemble` object from the forecast dataset
2. Adds the observation dataset as a reference
3. Computes valid times for each forecast initialization and lead time combination
4. Realigns the forecast dataset with explicit valid_time coordinates

The `HindcastEnsemble` framework enables consistent handling of forecast lead times across multiple initialization dates, crucial for calculating skill metrics.

8.3 Lead Time Calculation Example: JJA Season SPI

To illustrate the lead time concept, consider forecasting the June-July-August (JJA) SPI:

Forecast Month	Lead Time	Valid Period (SPI JJA)	Practical Usage
March	Lead time 4	June-July-August	Early planning
April	Lead time 3	June-July-August	Refinement
May	Lead time 2	June-July-August	Final decision

For example, forecasts initialized in March (Lead time 4) need three months to reach the start of the JJA period, providing earlier warning but typically with lower skill. Conversely, May forecasts (Lead time 2) offer higher accuracy but less preparation time.

The alignment process must account for these varying lead times, extracting the correct forecast-observation pairs:

```
# Select forecast for specific lead time
a_fc2 = a_fc1.isel(lead=params.lead_int)

# Find common dates between forecasts and observations
common_dates = np.intersect1d(fct_valid_times, obs_times)

# Filter observations to include only dates with matching forecasts
a_obs3 = a_obs2.sel(time=common_dates)

# Calculate initial dates by subtracting lead time from valid dates
a_fc3_init_dates = common_dates.astype("datetime64[M]") - np.timedelta64(
    int(a_fc3.lead.values), "M"
)

# Select forecasts initialized on these dates
a_fc4 = a_fc3.sel(init=a_fc3_init_dates)
```

This process creates perfect temporal alignment between forecast valid times and observation times, enabling direct comparison for verification statistics. It ensures that when calculating metrics like hit rates or false alarm ratios, the system is comparing forecasts and observations for exactly the same temporal periods.

8.4 Seasonal SPI Product Identification

Another sophisticated aspect of the alignment is identifying specific SPI seasonal products from the time coordinates:

```
# Create season identifiers (e.g., "MAM", "JJA")
spi_prod_list = spi3_prod_name_creator(a_fc2, "valid_time")
obs_spi_prod_list = spi3_prod_name_creator(a_obs, "time")

# Assign season identifiers as coordinates
a_fc2 = a_fc2.assign_coords(spi_prod=("init", spi_prod_list))
a_obs1 = a_obs.assign_coords(spi_prod=("time", obs_spi_prod_list))

# Filter to the specified season
a_fc3 = a_fc2.where(a_fc2.spi_prod == params.season_str, drop=True)
a_obs2 = a_obs1.where(a_obs1.spi_prod == params.season_str, drop=True)
```

This approach enables targeting specific three-month periods like MAM (March-April-May) or JJA (June-July-August), aligning with key agricultural seasons in East Africa while maintaining the temporal correspondence between forecast and observation datasets.

Chapter 9

Forecast Verification with xhistogram

9.1 Multi-dimensional Contingency Analysis

Forecast verification forms the cornerstone of any anticipatory action system, providing evidence that the forecasting system can reliably detect drought events before they occur. The `vthree_utils.py` script implements comprehensive verification methodology using the `xhistogram` library (imported as `xhist`), which enables efficient calculation of contingency tables and skill metrics across multiple dimensions.

At the core of this verification approach is the `xhist_metrics_2d` function, which performs grid-based verification of forecast probabilities against binary drought observations:

```
def xhist_metrics_2d(obs_data, ens_prob_data, params, calculate_auroc=True):
    """
    Calculates verification metrics for ensemble drought forecasts using
    multi-dimensional contingency tables.
    """
    # Get threshold for the specified region and season
    threshold_dict = get_threshold(params.region_id, params.sc_season_str)
    threshold = threshold_dict[params.level]

    # Process trigger values and calculate metrics
    df = process_trigger_values(
        obs_data, ens_prob_data, params, threshold, calculate_auroc
    )

    return df
```

This approach leverages multi-dimensional histograms to efficiently calculate contingency tables across the entire spatial domain, without requiring explicit loops over each grid cell.

9.2 Constructing Contingency Tables

The key innovation in this verification framework is the use of xarray's histogram functionality to construct contingency tables directly. This is implemented in the `calculate_contingency_table` function:

```
def calculate_contingency_table(obs_event, forecast_event, params):
    """Creates a 2x2 contingency table using xhistogram"""
    obs_event1 = obs_event[params.spi_prod_name]
    forecast_event1 = forecast_event[params.spi_prod_name]

    # Align forecast to observation time
```

```
    forecast_event3 = forecast_event1.assign_coords(
        init=forecast_event1.coords["valid_time"]
    )
    forecast_event3 = forecast_event3.rename({"init": "time"})

    # Histogram calculation produces the contingency table
    contingency_table = xhist.histogram(
        obs_event1, forecast_event3, bins=[2, 2], density=False, dim=["lat",
"lon"]
    )

    return contingency_table
```

This function:

1. Extracts the relevant SPI values from the observation and forecast datasets
2. Aligns the temporal coordinates of forecasts with observations
3. Computes a 2D histogram where each axis represents binary values (0=no drought, 1=drought)
4. Returns a contingency table with dimensions [time, obs_event, forecast_event]

The resulting contingency table has four elements representing the classic "hits", "misses", "false alarms", and "correct negatives" for each time step.

9.3 Verification Metrics Explained

From the contingency tables, a comprehensive set of verification metrics is calculated to assess different aspects of forecast quality:

1. **Hit Rate (HR)**:

   ```
   HR = \frac{hits}{hits + misses}
   ```

   Measures the proportion of observed drought events that were correctly forecast. Also called Probability of Detection (POD). Ranges from 0 (worst) to 1 (perfect).

2. **False Alarm Ratio (FAR)**:

   ```
   FAR = \frac{false\_alarms}{false\_alarms + hits}
   ```

   Measures the proportion of drought warnings that were false alarms. Ranges from 0 (perfect) to 1 (worst).

3. **Bias Score (BS)**:

   ```
   BS = \frac{hits + false\_alarms}{hits + misses}
   ```

   Indicates whether the system tends to under-forecast (BS < 1) or over-forecast (BS > 1) drought events.

4. **Hanssen-Kuipers Score (HK)**:

   ```
   HK = HR - \frac{false\_alarms}{false\_alarms + correct\_negatives}
   ```

   Also known as the True Skill Statistic (TSS), measures the ability to discriminate between drought and non-drought events. Ranges from -1 to 1.

5. **Heidke Skill Score (HSS)**:

   ```
   HSS = \frac{2(hits \cdot correct\_negatives - misses \cdot
   false\_alarms)}{(hits + misses)(misses + correct\_negatives) + (hits +
   false\_alarms)(false\_alarms + correct\_negatives)}
   ```

Measures the improvement of the forecast over random chance. Ranges from -∞ to 1.

6. **Area Under ROC Curve (AUROC)**:
   Measures the forecast's ability to discriminate between drought and non-drought events across all possible threshold values. Ranges from 0 to 1, with 0.5 indicating no skill.

9.4 AUROC and Bootstrapping

The AUROC calculation with bootstrapping represents the most sophisticated element of the verification methodology:

```
def calculate_auroc_score(contingency_table, trigger_value, n_bootstrap=1000):
    """
    Calculates the Area Under the ROC curve with confidence intervals
    using bootstrapping.
    """
    # Sum the contingency table across time
    summed_table = contingency_table.sum(dim="time")

    # Extract counts
    ct = summed_table.values.flatten()
    correct_negatives, false_alarms, misses, hits = ct
    total = hits + false_alarms + misses + correct_negatives

    # Perform bootstrapping
    auroc_bootstrap_scores = []
    for _ in range(n_bootstrap):
        bootstrap_counts = np.random.multinomial(
            total,
            [
                hits / total,
                misses / total,
                false_alarms / total,
                correct_negatives / total,
            ],
            size=1,
        )

        bootstrap_hits, bootstrap_misses, bootstrap_false_alarms,
 bootstrap_correct_negatives = bootstrap_counts[0]

        auroc_bootstrap_scores.append(
            calculate_auroc(
                bootstrap_hits,
                bootstrap_misses,
                bootstrap_false_alarms,
                bootstrap_correct_negatives,
            )
        )

    # Calculate mean and confidence intervals
    auroc_score = np.mean(auroc_bootstrap_scores)
    auroc_lb, auroc_ub = np.percentile(auroc_bootstrap_scores, [2.5, 97.5])

    return {
        "trigger_value": trigger_value,
        "auroc_score": auroc_score,
        "auroc_lb": auroc_lb,
        "auroc_ub": auroc_ub,
    }
```

This bootstrap approach:

1. Resamples from the original contingency table 1000 times
2. Calculates the AUROC score for each resample
3. Computes the mean AUROC score and 95% confidence intervals

Bootstrapping is crucial for drought verification because:

- Droughts are relatively rare events, leading to sampling uncertainty
- Confidence intervals provide stakeholders with a measure of uncertainty
- It helps identify whether skill scores are statistically significant

9.5 2D vs 1D Verification Methods

The system implements two complementary verification approaches:

1. **2D Verification (Grid-based)**:

```
def run_xhist2d(params):
    """Grid-based verification across the entire spatial domain"""
    # ...
    df = xhist_metrics_2d(obs_data, fct_mod, params, calculate_auroc=True)
    # ...
```

   This approach calculates metrics at each grid point and then aggregates the results, preserving the spatial structure of forecast skill. It's useful for understanding regional variations in forecast quality.

2. **1D Verification (Region-based)**:

```
def run_xhist1d(params):
    """Region-based verification using spatial means"""
    # ...
    obs_df = mean_obs_spi(obs_data, params.spi_prod_name)
    fct_df = mean_emp_prob(fct_mod, fct_sev, fct_ext, params.spi_prod_name)
    # ...
```

   This approach first calculates spatial means across the region and then performs verification on these mean values. It's useful for making operational decisions at the regional level.

Both approaches are necessary because:

- 2D verification captures the spatial heterogeneity of forecast skill
- 1D verification aligns better with decision-making at administrative levels
- Comparing both can identify issues with compensating errors in the forecasting system

The comprehensive verification framework implemented in these scripts provides the statistical basis for selecting optimal trigger values for anticipatory action, balancing the desire for high hit rates against the need to minimize false alarms within operational constraints.

Trigger Selection for Anticipatory Action against Droughts in East Africa

10.1 Forecast Verification and Trigger Selection Process

10.1.1 Data Processing Pipeline

1. **Initial Processing**
   - CHIRPS rainfall observations are processed to calculate SPI-3
   - SEAS5 precipitation forecasts are processed to calculate SPI-3 for different lead times
   - Both datasets are spatially aligned and masked to the region of interest (e.g., Karamoja)
2. **Verification Workflow**
   - `run_xhist2d.py` performs 2D verification by comparing ensemble SPI forecasts against observed SPI
   - `run_xhist1d.py` performs 1D verification by comparing mean ensemble SPI against regional mean SPI
   - Results are saved as CSV files for different lead times, regions, and seasons

10.1.2 Critical Manual Step: Trigger Selection

The most critical manual step in the workflow is **trigger selection**, which occurs after verification metrics are calculated. This process:

1. Examines the CSV files generated by `run_xhist2d.py` and `run_xhist1d.py`
2. Identifies optimal trigger values based on verification metrics
3. Manually marks selected triggers by adding a value of '1' in the `td` column

For example, in the CSV file for Karamoja MAM season with lead time 3:

```
tv year# hr far bs hk hs au odc h m FA CN h% cat td
15.2 10 96.9 1.6 1.0 -0.0 -0.0 1.0 13 12 1 27 2 92.3 mild 0.0
...
68.0 21 72.7 0.0 0.7 nan 0.0 0.7 8 3 5 17 17 37.5 mod 1.0
...
```

The row with `td=1.0` indicates the manually selected trigger value (68% probability) for the moderate category.

10.1.3 Selection Criteria

Triggers are selected based on a combination of metrics:

- **AUROC** (Area Under ROC Curve) > 0.5 (skill better than random)
- **Hit Rate** (HR) > 50% (minimum detection rate)
- **False Alarm Ratio** (FAR) < 35% (maximum acceptable false alarms)
- **Operational considerations** (lead time, practical implementation)

10.2 Verification Metrics

10.2.1 2D Verification Metrics

- Used for spatial analysis of forecast skill
- Calculates contingency table statistics by gridcell
- Produces `auroc_scores`, `hit_rates`, and `false_alarm_ratios`

10.2.2 1D Verification Metrics

- Analyzes regional mean forecast performance
- Used for operational decision-making
- Focuses on practical application of triggers
- Metrics include:
  - `hit_rate`: Percent of actual drought events correctly forecast
  - `false_alarm_ratio`: Proportion of forecast events that didn't occur
  - `bias_score`: Ratio of forecast frequency to observed frequency
  - `hanssen_kuipers_scores`: Difference between hit rate and false alarm rate
  - `heidke_skill_scores`: Forecast accuracy relative to random chance

10.3 Downstream Processing Dependencies

The manually selected triggers (marked with `td=1`) are essential inputs for:

1. ``**run_bar_plot.py**``
   - Creates comparative bar visualizations
   - Shows observed SPI values alongside forecast probabilities
   - Highlights performance based on selected trigger values
2. ``**run_heatmap.py**``
   - Generates heatmaps of hit rates and false alarm ratios
   - Visualizes performance across different months and lead times
   - Supports seasonal decision-making
3. ``**run_latex_table.py**``

- Produces formatted tables of available triggers
- Highlights selected triggers
- Documents full decision metrics for stakeholders

10.4 Forecast Skill Assessment

For each potential trigger, the system calculates:

- Number of hits (h): Correctly forecast drought events
- Number of misses (m): Unforecast drought events
- Number of false alarms (FA): Forecast events that didn't occur
- Number of correct negatives (CN): Correctly forecast non-events
- Hit percentage (h%): Percentage of observed events successfully forecast

10.5 Operational Implementation

The selected triggers enable a three-phase anticipatory action approach:

1. **Ready**: Initial alert based on early forecasts
2. **Set**: Confirmation with subsequent forecasts
3. **Go**: Implementation of anticipatory actions

The triggers serve as decision thresholds for initiating the anticipatory action workflow, with different trigger values potentially applied based on vulnerability levels (general vs. emergency menus).

10.6 Time Window Considerations

Triggers are optimized for specific seasonal windows:

- **Window 1**: Start to mid-season (e.g., MAM for Karamoja)
- **Window 2**: Middle to end of season (e.g., JJA for Karamoja)

This temporal approach allows for targeted interventions at critical points in the agricultural cycle.

Map Visualization for Ensemble Forecasts and Observations

This document explains how to create map visualizations from ensemble forecasts and observational data for drought monitoring using Python GIS tools.

11.1 Setting Up the Environment

First, import the necessary libraries:

```
import xarray as xr
import numpy as np
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import geopandas as gpd
import os
from PIL import Image
from shapely import wkb
```

11.2 Data Organization with DataTree

DataTree provides a hierarchical structure for organizing ensemble data efficiently:

```
def helper_stamp_plot(ens_data, obs_data, fct_mod, fct_sev, fct_ext):
    """
    Helper function to organize data into a tree structure.

    Parameters:
    ens_data (xarray.Dataset): Ensemble data
    obs_data (xarray.Dataset): Observation data
    fct_mod, fct_sev, fct_ext (xarray.Dataset): Forecast datasets
    """
    seas51tree = xr.DataTree()

    # Add ensemble members
    for member in ens_data.member:
        member_data = ens_data.sel(member=member)
        seas51tree[f"ensemble/member_{int(member)}"] = xr.DataTree(
            name=f"member_{int(member)}",
            dataset=member_data
        )
```

Add observation and forecast datasets to the DataTree:

```
# Add observation and forecast datasets
```

```
seas51tree["observation"] = xr.DataTree(
    name="observation",
    dataset=obs_data
)
seas51tree["fct_mod"] = xr.DataTree(
    name="fct_mod",
    dataset=fct_mod
)
seas51tree["fct_sev"] = xr.DataTree(
    name="fct_sev",
    dataset=fct_sev
)
seas51tree["fct_ext"] = xr.DataTree(
    name="fct_ext",
    dataset=fct_ext
)

return seas51tree
```

11.3 Creating Single-Row Time Series Plot

The following function creates visualizations for a specific initialization date:

```
def create_single_row_plot(tree, init, params, region_geom):
    """
    Create a row of maps for a specific initialization date.

    Parameters:
    tree (xarray.DataTree): Data organized in tree structure
    init (datetime): Initialization date
    params (object): Parameters for plotting
    region_geom (geometry): Region geometry for overlay
    """
    members = list(tree["ensemble"].children.keys())
    valid_times = tree["ensemble/member_0"].ds.valid_time.values
    num_members = len(members)
    num_additional_plots = 4  # Obs, mod, sev, ext
    total_plots = num_members + num_additional_plots
```

Set up the figure and define color scales:

```
# Create figure with map projections
fig, axs = plt.subplots(
    1, total_plots,
    figsize=(2 * total_plots, 2),
    subplot_kw={"projection": ccrs.PlateCarree()}
)

# Define color scales
ensemble_cmap_range = (-4, 4)
fct_cmap_range = (0.0, 1.0)

# Get valid time for this initialization
valid_time = valid_times[
    np.where(tree["ensemble/member_0"].ds.init.values == init)[0][0]
]
```

Plot ensemble members and additional data:

```
# Plot ensemble members
```

```
for j, member_key in enumerate(members):
    _plot_ensemble_member(
        tree, member_key, params.spi_prod_name, init,
        axs[j], ensemble_cmap_range, region_geom,
        shape_overlay=(j == 0)  # Only show shapefile on first plot
    )

# Add observation and forecast plots
plot_titles = ["Obs", "mod", "sev", "ext"]
plot_keys = ["observation", "fct_mod", "fct_sev", "fct_ext"]

for k, (title, key) in enumerate(zip(plot_titles, plot_keys)):
    _plot_additional_data(
        tree, key, params.spi_prod_name, init, valid_time,
        axs[num_members + k], title,
        ensemble_cmap_range if key == "observation" else fct_cmap_range
    )

return fig, axs, (2 * total_plots, 2)
```

11.4 Plotting Map Elements

The system uses helper functions to plot ensemble members:

```
def _plot_ensemble_member(tree, member_key, variable, init, ax, cmap_range,
                          geom, shape_overlay=False):
    """Plot individual ensemble member."""
    member_data = tree[f"ensemble/{member_key}"].ds[variable]
    data = member_data.sel(init=init).values
    time_np64 = np.array(str(init), dtype="datetime64[ns]")
    year = pd.to_datetime(time_np64).year

    # Plot data
    ax.pcolormesh(
        tree["ensemble/member_0"].ds.lon.values,
        tree["ensemble/member_0"].ds.lat.values,
        data, cmap="RdBu", transform=ccrs.PlateCarree(),
        vmin=cmap_range[0], vmax=cmap_range[1]
    )
```

Add title, formatting, and optional shapefile overlay:

```
# Add title and formatting
ax.set_title(f'{year}m{member_key.split("_")[1]}', fontsize=6)
ax.set_xticks([])
ax.set_yticks([])

# Overlay shapefile if requested
if shape_overlay:
    ax.add_geometries(
        [geom], crs=ccrs.PlateCarree(),
        edgecolor="black", facecolor="none"
    )
```

Helper function to plot observation or forecast data:

```
def _plot_additional_data(tree, key, variable, init, valid_time, ax,
                          title, cmap_range):
    """Plot observation or forecast data."""
    dataset = tree[key].ds[variable]
```

```
    # Handle different coordinate naming conventions
    coord_key = "time" if "time" in dataset.coords else "init"
    obs_init = (
        np.datetime64(valid_time.strftime("%Y-%m-%d %H:%M:%S"))
        if coord_key == "time" else init
    )

    # Extract data
    obs_data = dataset.sel({coord_key: obs_init}).values
```

Set appropriate colormap and plot data:

```
# Set appropriate colormap
cmap = "RdBu" if key == "observation" else "Blues"

# Plot data
ax.pcolormesh(
    tree["ensemble/member_0"].ds.lon.values,
    tree["ensemble/member_0"].ds.lat.values,
    obs_data, cmap=cmap, transform=ccrs.PlateCarree(),
    vmin=cmap_range[0], vmax=cmap_range[1]
)

# Add title and formatting
ax.set_title(title, fontsize=6)
ax.set_xticks([])
ax.set_yticks([])
```

11.5 Processing Multiple Initializations

To create a complete visualization, the system processes multiple initialization dates:

```
def plot_allrows(seas51tree, shapefile_df, params):
    """
    Create plots for all initialization dates and combine them.

    Parameters:
    seas51tree (xarray.DataTree): Data in tree structure
    shapefile_df (GeoDataFrame): Shapefile data
    params (object): Plot parameters
    """
    # Create output directory
    output_dir = (f"{params.output_path}map_{params.region_id}_"
                  f"{params.sc_season_str}_lt{params.lead_int}")
    os.makedirs(output_dir, exist_ok=True)

    # Get region geometry
    region_geom = shapefile_df["geometry"].values[0]

    # Get all initialization dates
    inits = seas51tree["ensemble/member_0"].ds.init.values
    plot_files = []
```

Create plots for each initialization date:

```
# Process each initialization date
for i, init in enumerate(inits):
    fig, axs, figsize = create_single_row_plot(
        seas51tree, init, params, region_geom
```

```
    )

    # Save individual plot
    output_file = f'{output_dir}/stamp_plot_{init.strftime("%Y%m%d")}.png'
    plot_files.append(output_file)
    fig.savefig(output_file, dpi=100, bbox_inches="tight")
    plt.close(fig)

# Add colorbar and title to the final figure
add_colorbar_and_title(figsize, params, output_dir)

# Merge all plots into a single image
merge_png_files(
    input_dir=output_dir,
    output_file=(f"map_{params.region_id}_{params.sc_season_str}_"
                 f"lt{params.lead_int}.pdf"),
    delete_originals=False
)
```

11.6 Merging Individual Plots

The following function combines individual timestep plots into a single visualization:

```
def merge_png_files(input_dir, output_file, delete_originals=False):
    """
    Merge individual PNG files into a single image.

    Parameters:
    input_dir (str): Directory containing PNG files
    output_file (str): Name of output file
    delete_originals (bool): Whether to delete original files
    """
    # Get all PNG files
    png_files = sorted(Path(input_dir).glob("*.png"))

    # Open all images and get dimensions
    images = []
    max_width = 0
    total_height = 0
```

Process images and create a new composite:

```
for png_file in png_files:
    with Image.open(png_file) as img:
        images.append(img.copy())
        max_width = max(max_width, img.width)
        total_height += img.height

# Create new composite image
merged_image = Image.new("RGB", (max_width, total_height), (255, 255, 255))

# Paste each image into the merged image
y_offset = 0
for img in images:
    merged_image.paste(img, ((max_width - img.width) // 2, y_offset))
    y_offset += img.height

# Save merged image
output_path = os.path.join(input_dir, output_file)
merged_image.save(output_path, dpi=(300, 300))
```

Optionally delete original files:

```
# Optionally delete original files
if delete_originals:
    for png_file in png_files:
        os.remove(png_file)
```

11.7 Example Usage

To run the map visualization process:

```
def run_map_plot(params, use_local=False):
    """
    Run the complete mapping process.

    Parameters:
    params (object): Parameters for the run
    use_local (bool): Whether to use local files

    Returns:
    str: Status message
    """
    # Get thresholds for the region and season
    threshold_dict = get_threshold(params.region_id, params.sc_season_str)

    # Prepare datasets
    obs_data, ens_data = make_obs_fct_dataset(params)
```

Process data and create plots:

```
# Calculate empirical probabilities
fct_mod, fct_sev, fct_ext = seas51_patch_empirical_probability(
    ens_data, threshold_dict
)

# Create data tree
dstree = helper_stamp_plot(ens_data, obs_data, fct_mod, fct_sev, fct_ext)

# Get region bounds
if use_local:
    shapefile_df, extent = get_region_bounds(region_id, use_local=True)
else:
    credentials = get_credentials('coiled-data.json')
    shapefile_df, extent = get_region_bounds(
        region_id, credentials, use_local=False, buffer=0.5
    )

# Create and merge plots
plot_allrows(dstree, shapefile_df, params)
```

Create final merged PDF:

```
# Create final merged PDF
merge_png_files(
    input_dir=(f"{params.output_path}map_{params.region_id}_"
               f"{params.sc_season_str}_lt{params.lead_int}"),
    output_file=(f"map_{params.region_id}_{params.sc_season_str}_"
                 f"lt{params.lead_int}.pdf"),
    delete_originals=False
)
```

```
return "merge plot"
```

11.8 Conclusion

This approach allows for the effective visualization of both ensemble forecasts and observed data on maps with region overlays. The plots are organized by initialization date, allowing stakeholders to track forecast evolution over time and make informed decisions about anticipatory actions for drought mitigation.